

P3179R0

C++ parallel range algorithms

Published Proposal, 2024-03-14

This version:

<https://wg21.link/P3179R0>

Authors:

[Ruslan Arutyunyan](#) (Intel)

[Alexey Kukanov](#) (Intel)

Audience:

SG9, SG1

Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Abstract

This paper proposes adding parallel algorithms that work together with the C++ Ranges library.

Table of Contents

1 Motivation

2 Design overview

2.1 Design summary

2.2 Coexistence with schedulers

2.3 Switch to parallel range algorithms with minimal changes

2.4 Algorithm return types

2.5 Non ADL-discoverable functions

2.6 Requiring `random_access_iterator` or `random_access_range` for parallel policies

2.7 `const`-callable parameter

2.8 Parallel range algorithms are not customization points

2.9 Parallel range algorithms are not `constexpr`

3 More examples

3.1 Less parallel algorithm calls and better expressiveness

4 Proposed API

4.1 Possible implementation of a parallel range algorithm

5 Further exploration

5.1 Thread-safe views examination

5.2 Absence of some serial range-based algorithms

5.3 Output for parallel range algorithms

References

Informative References

§ 1. Motivation

Standard parallel algorithms with execution policies which set semantic requirements to user-provided callable objects were a good start for supporting parallelism in the C++ standard.

The C++ Ranges library - ranges, views, etc. - is a powerful facility to produce lazily evaluated pipelines that can be processed by range-based algorithms. Together they provide a productive and expressive API with the room for extra optimizations.

Combining these two powerful features by adding support for execution policies to the range-based algorithms opens an opportunity to fuse several computations into one parallel algorithm call, thus reducing the overhead on parallelism. That is especially valuable for heterogeneous implementations of parallel algorithms, for which the range-based API helps reducing the number of kernels submitted to an accelerator.

Earlier, [\[P2500R2\]](#) proposed to add the range-based C++ parallel algorithms together with its primary goal of extending algorithms with schedulers. We have decided to split those parts to separate papers, which could progress independently.

This paper is targeted to C++26.

§ 2. Design overview

This proposal addresses absence of execution policy support for C++ range-based algorithms. In the nutshell, the proposal extends C++ range algorithms with overloads taking any standard C++ execution policy as a function parameter. These overloads are further referred to as *parallel range algorithms*.

§ 2.1. Design summary

- The parallel range algorithms should be close to C++17 classic ones to use the code with minimal required changes.
- The parallel range algorithms should return the same type as the corresponding serial range algorithms.
- The proposed algorithms should be special non-ADL discoverable functions, same as serial range algorithms.
- The required range and iterator categories should at least be random access for all but `std::execution::seq` execution policies.
- The proposed API should require any callable object passed to an algorithm to have `const`-qualified `operator()`.
- The proposed API is not a customization point.
- The proposed API is not `constexpr`.

§ 2.2. Coexistence with schedulers

We believe that adding parallel range algorithms does not have the risk of conflict with anticipated scheduler-based algorithms, because an execution policy does not satisfy the requirements for a policy-aware scheduler ([P2500R2]), a sender ([P3300R0]), or really anything else from [P2300R7] that can be used to specify such algorithms.

At this point we do not, however, discuss how the appearance of schedulers may or should impact the execution rules for parallel algorithms specified in [\[algorithms.parallel.exec\]](#), and just assume that the same rules apply to the range algorithms with execution policies.

§ 2.3. Switch to parallel range algorithms with minimal changes

One of the goals is to require a minimal amount of changes when switching from the existing API to parallel range algorithms.

The C++17 parallel `for_each` call:

```
std::for_each(std::execution::par, v.begin(), v.end(), [](auto& x) { ++x; });
```

can be changed to one of the following:

```
// Using an iterator and a sentinel
std::ranges::for_each(std::execution::par, v.begin(), v.end(), [](auto& x) { ++x; });

// Switching to use a range
std::ranges::for_each(std::execution::par, v, [](auto& x) { ++x; });
```

If serial range algorithms are used in the code, switching to parallel version would look like in the example below.

The C++20 range-based `for_each` call:

```
std::ranges::for_each(v, [](auto& x) { ++x; });
```

becomes:

```
std::ranges::for_each(std::execution::par, v, [](auto& x) { ++x; });
```

As you can see the changes are pretty simple:

- In the first case only the namespace is changed, and users might also change `v.begin()`, `v.end()` to just `v`.
- In the second case an execution policy is added as the first function argument. The same is true for the *Iterator + Sentinel* overload

§ 2.4. Algorithm return types

We explored possible algorithm return types and came to conclusion that returning the same type as serial range algorithms is the preferred option to make the changes for enabling parallelism minimal.

We can consider the same example as in the previous paragraph.

```
// for_each_result<I, Fun> may be used instead of auto
auto res = std::ranges::for_each(v, [](auto& x) { ++x; });
```

becomes:

```
// for_each_result<I, Fun> may be used instead of auto
auto res = std::ranges::for_each(std::execution::par, v, [](auto& x) { ++x; });
```

§ 2.5. Non ADL-discoverable functions

We believe the proposed functions should be non-ADL discoverable same as serial range algorithms. Whether a serial version is implemented with the special compiler support or as a global callable object, adding overloads for the parallel version should not be a problem. Please see [§ 4.1 Possible implementation of a parallel range algorithm](#) for more information.

§ 2.6. Requiring `random_access_iterator` or `random_access_range` for parallel policies

C++17 parallel algorithms require *LegacyForwardIterator* for input data sequences. Although it might be useful for `std::execution::seq` policy, it does not make a lot of sense for an actual parallel implementation. We are not aware of an existing implementation supporting forward iterators well for any of `unseq`, `par` or `par_unseq` policies. oneAPI Data Parallel C++ library (oneDPL) supports forward iterators only for a very few algorithms, only for `par` and only in the implementation based on oneTBB.

We think that requiring `random_access_iterator` or `random_access_range` for `unseq`, `par`, and `par_unseq` policies will make the user experience better because it sets the right expectations from the very beginning. The `seq` policy would still require `forward_iterator` or `forward_range`.

§ 2.7. `const`-callable parameter

We believe that parallel range algorithms should require function objects for predicates, comparators, functions expected by `for_each`, etc. to be `const`-callable. It seems important to add that requirement for the extra safety it gives when parallelism is introduced to existing serial code, providing compile-time diagnostics for mutable function objects which might be unsafe for parallel execution.

The following example works fine for serial code. While it compiles for parallel code, users should not assume that the semantics remains intact. Since the parallel version of `for_each` requires function object to be copyable, it is not guaranteed that all `for_each` iterations are processed by the same function object. Practically speaking, users cannot rely on accumulating any state modifications in a parallel `for_each` call.

```
struct callable
{
    void operator()(int& x)
    {
        ++x;
        ++i; // race here for parallel code
    }
    int get_i() const {
        return i;
    }
private:
    int i = 0;
};

callable c;

// serial for_each call
auto fun = std::for_each(v.begin(), v.end(), c);

// parallel for_each call
// The callable object cannot be read because parallel for_each version pulls
std::for_each(std::execution::par, v.begin(), v.end(), c);

// for_each serial range version call
auto [_, fun] = std::ranges::for_each(v.begin(), v.end(), c);
```

As we stated above we would like to preserve the return types of serial range algorithms, therefore the parallel `std::ranges::for_each` still returns a copy of the callable object. However, we propose it to fail to compile if the `operator()` of the callable is not marked as `const`:

```
// callable is used from the previous code snippet
// Fails to compile with our proposal because callable::operator() is not const
callable c;
auto [_, fun] = std::ranges::for_each(std::execution::par, v.begin(), v.end());
```

Of course, that requirement is easy to overcome by wrapping the callable object. In that case, it is user's responsibility to make sure that the code is free from data races. Please see the example below:

```
// callable is used from the previous code snippet
// Wrapping a callable objection with std::reference_wrapper compiles, but
callable c;
auto [_, fun] = std::ranges::for_each(std::execution::par, v.begin(), v.end());
```

§ 2.8. Parallel range algorithms are not customization points

We do not propose the parallel range algorithms to be customization points because it's unclear which parameter to customize for. One could argue that customizations may exist for execution policies, but we expect custom execution policies to become unnecessary once the C++ algorithms will work with schedulers/senders/receivers.

§ 2.9. Parallel range algorithms are not `constexpr`

We do not propose the new parallel algorithms to be `constexpr`. We are aware of [\[P2902R0\]](#) and might align with it in the future, however we don't think that making parallel algorithms `constexpr` should be a primary design goal.

§ 3. More examples

§ 3.1. Less parallel algorithm calls and better expressiveness

Let's consider the following example:

```
reverse(policy, begin(data), end(data));  
transform(policy, begin(data), end(data), begin(result), [](auto i){ return  
auto res = any_of(policy, begin(result), end(result), pred);
```

It has three stages and eventually tries to answer the question if the input sequence contains an element after reversing and transforming it. The interesting considerations are:

- Since the example has three parallel stages, it adds extra overhead for parallel computation per algorithm.
- The first two stages will complete for all elements before the `any_of` stage is started, though it is not required for correctness. If reverse and transformation would be done on the fly, a good implementation of `any_of` might have skipped the remaining work when `pred` returns `true`, thus providing more performance.

Let's make it better:

```
// With fancy iterators  
auto res = any_of(policy,  
    make_transform_iterator(make_reverse_iterator(end(data)),  
        [](auto i){ return i * i; })),  
    make_transform_iterator(make_reverse_iterator(begin(data)),  
        [](auto i){ return i * i; })),  
    pred);
```

Now there is only one parallel algorithm call, and `any_of` can skip unneeded work. However, this variation also has interesting considerations:

- First, it doesn't compile. We use `transform_iterator` to pass the transformation function, but the two `make_transform_iterator` expressions use two different lambdas, and the iterator type for `any_of` cannot be deduced because the types of `transform_iterator` do not match. One of the options to make it compile is to store a lambda in a variable.

- Second, it requires using a non-standard iterator.
- Third, the expressiveness of the code is not good: it is hard to read while easy to make a mistake like the one described in the first bullet.

Let's improve the example further with the proposed API:

```
// With ranges
auto res = any_of(policy, data | views::reverse | views::transform([](auto
    pred));
```

The example above lacks the drawbacks described for the previous variations:

- There is only one algorithm call;
- The implementation might skip unnecessary work;
- There is no room for the lambda type mistake;
- The readability is much better compared to the second variation and not worse than in the first one.

§ 4. Proposed API

NOTE: `std::ranges::for_each` is used as a reference point. When the design is ratified, it will be spread across other algorithms.

```
// Policy-based API
template <class ExecutionPolicy, policy-dependent-iterator I, sentinel_for<I> S,
    class Proj = identity, indirectly_unary_invocable<projected<I, Proj>> Fun>
    ranges::for_each_result<I, Fun>
    ranges::for_each(ExecutionPolicy&& policy, I first, S last, Fun f, Proj proj = {});

template <class ExecutionPolicy, policy-dependent-range R, class Proj = identity,
    indirectly_unary_invocable<projected<iterator_t<R>, Proj>> Fun>
    ranges::for_each_result<ranges::borrowed_iterator_t<R>, Fun>
    ranges::for_each(ExecutionPolicy&& policy, R&& r, Fun f, Proj proj = {});
```

`policy-dependent-iterator` and `policy-dependent-range` are exposition only concepts defined as:

- If `ExecutionPolicy` type is `std::execution::sequenced_policy` then `policy-dependent-iterator` is the `forward_iterator` concept and `policy-dependent-range` is the `forward_range` concept.
- Otherwise, `policy-dependent-iterator` is the `random_access_iterator` concept and `policy-dependent-range` is the `random_access_range` concept.

§ 4.1. Possible implementation of a parallel range algorithm

```
// A possible implementation of std::ranges::for_each
namespace ranges
{
    namespace __detail
    {
        struct __for_each_fn
        {
            // ...
            // Existing serial overloads
            // ...

            // The dedicated overload for sequenced_policy. Requires forward_iterator
            template<forward_iterator I, sentinel_for<I> S,
                    class Proj = identity, indirectly_unary_invocable<projected<I, Proj>> F>
            ranges::for_each_result<I, Fun>
            operator()(const std::execution::sequenced_policy&, I first, S last, F f,
                    Proj proj = {}) const
            {
                for (; first != last; ++first)
                {
                    std::invoke(f, std::invoke(proj, *first));
                }
                return {std::move(first), std::move(f)};
            }

            // The overload for unsequenced and parallel policies. Requires random_access_iterator
            template<class ExecutionPolicy, random_access_iterator I, sentinel_for<I> S,
                    class Proj = identity, indirectly_unary_invocable<projected<I, Proj>> F>
            requires is_execution_policy_v<std::remove_cvref_t<ExecutionPolicy>>
            ranges::for_each_result<I, Fun>
            operator()(const ExecutionPolicy&, I first, S last, F f,
                    Proj proj = {})
```

```

ranges::for_each_result<I, Fun>
operator()(ExecutionPolicy&& exec, I first, S last, Fun f, Proj proj =
{
    // properly handle execution policy; for the reference, a serial
    // implementation is provided
    for (; first != last; ++first)
    {
        std::invoke(f, std::invoke(proj, *first));
    }
    return {std::move(first), std::move(f)};
}

template<class ExecutionPolicy, forward_range R, class Proj = identity,
        indirectly_unary_invocable<projected<iterator_t<R>, Proj>> Fun>
ranges::for_each_result<ranges::borrowed_iterator_t<R>, Fun>
operator()(ExecutionPolicy&& exec, R&& r, Fun f, Proj proj = {}) const
{
    return (*this)(std::forward<ExecutionPolicy>(exec), std::ranges::begin(
        std::ranges::end(r), f, proj);
}
}; // struct for_each
} // namespace __detail
inline namespace __for_each_fn_namespace
{
    inline constexpr __detail::__for_each_fn for_each;
} // __for_each_fn_namespace
} // namespace ranges

```

§ 5. Further exploration

§ 5.1. Thread-safe views examination

We need to understand better whether using some `views` with parallel algorithms might result in data races. At first glance, requiring iterators and ranges to provide random access should be sufficient to prevent such issues, but we want to be sure.

§ 5.2. Absence of some serial range-based algorithms

We understand that some useful algorithms, for example, most of generalized numeric operations [[numeric.ops](#)] do not yet exist in `std::ranges` even in a serial version. It is supposed to be addressed either by this or by a complementary paper.

§ 5.3. Output for parallel range algorithms

Serial range algorithms take only `output_iterator` as the result. In other words, there is no overload (for example, for `copy_if` algorithm) that takes both input and output as `range`. We would like to explore whether it's worth adding such an overload for parallel range algorithms because it might be more useful to have both input and output as ranges, for safety and performance reasons.

`copy_if` is a good example also because it doesn't require both input and output sequence to have the same size. But for parallelization purpose it is useful to know the size of passed sequences in advance. If the output for `copy_if` is an iterator, not a range we don't know the output sequence size and we cannot rely on the input sequence size.

§ References

§ Informative References

[P2300R7]

Eric Niebler, Michał Dominiak, Georgy Evtushenko, Lewis Baker, Lucian Radu Teodorescu, Lee Howes, Kirk Shoop, Michael Garland, Bryce Adelstein Lelbach. [`std::execution`](#). 21 April 2023. URL: <https://wg21.link/p2300r7>

[P2500R2]

Ruslan Arutyunyan, Alexey Kukanov. [C++ parallel algorithms and P2300](#). 15 October 2023. URL: <https://wg21.link/p2500r2>

[P2902R0]

Oliver Rosten. [constexpr 'Parallel' Algorithms](#). 17 June 2023. URL: <https://wg21.link/p2902r0>

[P3300R0]

Bryce Adelstein Lelbach. [C++ Asynchronous Parallel Algorithms](#). URL: <https://wg21.link/p3300r0>